Technical Home Assignment – Industrial / Real-Time System

This assignment evaluates your ability to build a real-time, distributed system similar to software used in industrial digital press environments. This document defines requirements only.

1. Mandatory Rules

- AI-assisted development is mandatory (Cursor or equivalent).
- All AI prompts must be documented under the /prompts folder.
- The full system must run using docker-compose.
- All mandatory requirements must be met exactly as written.

2. Required System Components

- React + TypeScript UI with exactly 3 pages
- REST API service (C# or Python)
- SQL Data service (C# + Entity Framework)
- IoT Telemetry service (C# or C++)
- Redis, RabbitMQ, SQL Database

3. Real-Time & Sensor Requirements

- The system must simulate exactly 20 sensors.
- Each sensor must generate telemetry once per second.
- Telemetry must originate in Redis.
- Real-time telemetry must be pushed to the UI via SignalR.
- Polling for telemetry updates is not allowed.

4. Communication Constraints

- Backend services must communicate exclusively using gRPC and RabbitMQ.
- SignalR is allowed only between the REST API service and the UI.
- REST is allowed only between the UI and the REST API.
- Any other communication patterns are forbidden.

5. Testing Requirements

- Unit tests are required for all backend services.
- Integration tests must validate end-to-end behavior.
- Integration tests must prove real-time flow for all 20 sensors.
- All tests must execute successfully in CI.

6. Architecture, Scaling & Performance Explanation (Mandatory)

Include a section in your README.md that explains:

- Your high-level system architecture and service boundaries.
- How data flows through the system (SQL → API, Redis → API → UI).

- Why gRPC, RabbitMQ, and SignalR are used where they are.
- Key trade-offs and constraints you considered.
- How the architecture would evolve to support more sensors or higher update rates.
- How the system behaves in failure scenarios (e.g. Redis unavailable, RabbitMQ delayed, service restarts).
- What logs, metrics, or operational signals you would add to debug and operate this system in production.
- How your design handles different sensor behaviors (e.g. sensors that change very frequently, very slowly, or remain constant for long periods), and what you would optimize if performance became a concern.
- Which parts of the system you consider stable versus likely to change.
- Whether you would implement the solution differently, and if so, how and why (alternative designs and improvements).

7. Docker & CI/CD Requirements

Each service must include a Dockerfile. A docker-compose.yml must be provided to run the full system. CI/CD must be implemented using GitHub Actions or Azure DevOps Pipelines and must build services, run tests, and build Docker images.

8. Documentation & Submission

- Git repository URL.
- Working docker-compose setup.
- Passing CI pipeline.
- README covering setup, tests, and architecture explanation.
- Complete AI prompt documentation.